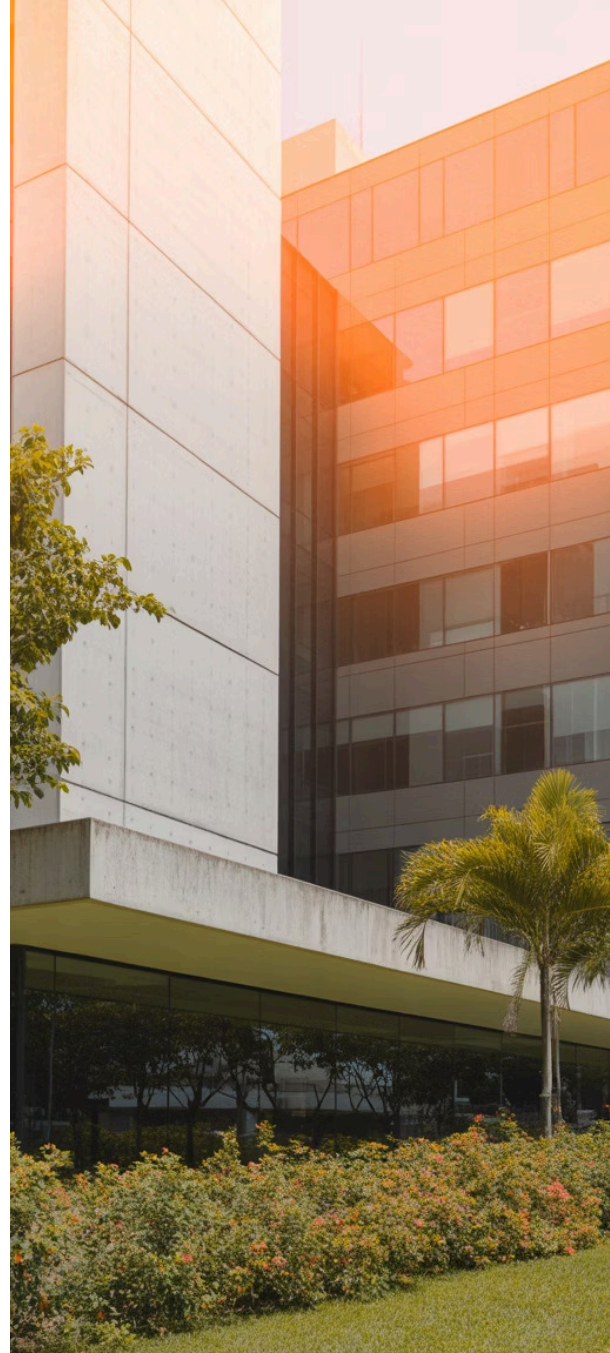


Relacionamentos entre Classes – Composição

UNIVERSIDADE DO OESTE DE SANTA CATARINA - UNOESC

Ciência da Computação | Programação II

Prof.: Leandro Otavio Cordova Vieira





Objetivos da Aula

1 Compreender o conceito de composição

Entender os fundamentos teóricos da composição como um tipo especial de relacionamento entre classes, onde existe uma dependência existencial forte entre o todo e suas partes.

2 Diferenciar composição de associação e agregação

Identificar as características distintivas entre os três tipos de relacionamentos, focando especialmente nas nuances que tornam a composição única e mais restritiva.

3 Aplicar em cenários reais com PHP

Implementar exemplos práticos de composição utilizando PHP, demonstrando como o conceito teórico se traduz em código funcional e estruturado.

4 Desenvolver atividade prática orientada

Participar de exercícios práticos que consolidem o aprendizado através da modelagem e implementação de sistemas que utilizam composição de forma adequada.

Revisão Rápida

Antes de mergulharmos na composição, vamos relembrar os conceitos fundamentais dos relacionamentos entre classes que já estudamos anteriormente.

Associação

Representa uma **relação simples** entre classes, onde objetos de uma classe conhecem e podem interagir com objetos de outra classe, mas mantêm sua independência completa.

- Relação mais fraca
- Objetos independentes
- Pode existir sem o outro

Agregação

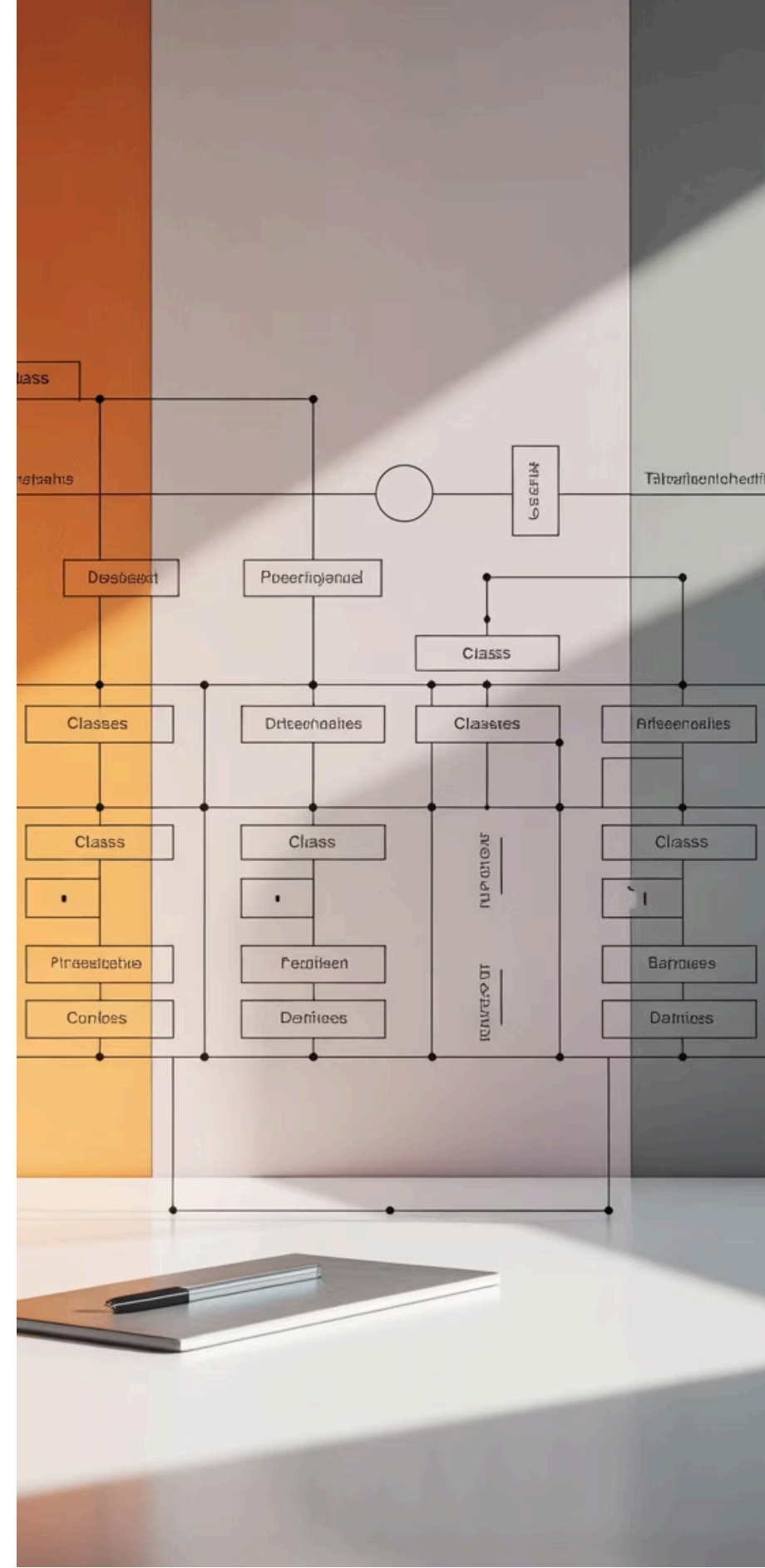
Estabelece uma **relação fraca "todo-parte"**, onde uma classe contém outras, mas as partes podem existir independentemente do todo.

- Relação "tem um"
- Partes independentes
- Diamante vazio na UML

Composição

Define uma **relação forte "todo-parte"**, onde as partes dependem completamente do todo para sua existência e ciclo de vida.

- Relação mais forte
- Dependência existencial
- Diamante preenchido na UML





O que é Composição?

A composição representa o relacionamento mais forte entre classes na programação orientada a objetos. É caracterizada por uma dependência existencial absoluta entre o todo e suas partes.

Responsabilidade Existencial

O objeto "todo" é **completamente responsável** pela criação, gerenciamento e destruição de suas partes componentes.

Dependência Vital

As partes **não podem existir** sem o todo. Quando o objeto principal é destruído, todas as suas partes também são automaticamente eliminadas.

Ciclo de Vida Compartilhado

O nascimento e morte das partes estão **intrinsecamente ligados** ao ciclo de vida do objeto que as contém.

❏ **Importante:** Na composição, a parte nunca pode ser compartilhada com outros objetos "todo". Cada parte pertence exclusivamente a um único objeto principal.

Diferenças-Chave

Compreender as distinções entre os três tipos de relacionamentos é fundamental para aplicá-los corretamente em seus projetos.

Aspecto	Associação	Agregação	Composição
Tipo de Vínculo	Externo e opcional	Interno mas flexível	Interno e obrigatório
Independência	Totalmente independentes	Partes independentes	Dependência existencial total
Ciclo de Vida	Ciclos separados	Ciclos podem diferir	Ciclo compartilhado
Responsabilidade	Cada um por si	Gerenciamento opcional	Todo controla tudo
Exemplo	Professor-Aluno	Departamento-Funcionário	Casa-Cômodo

Representação UML

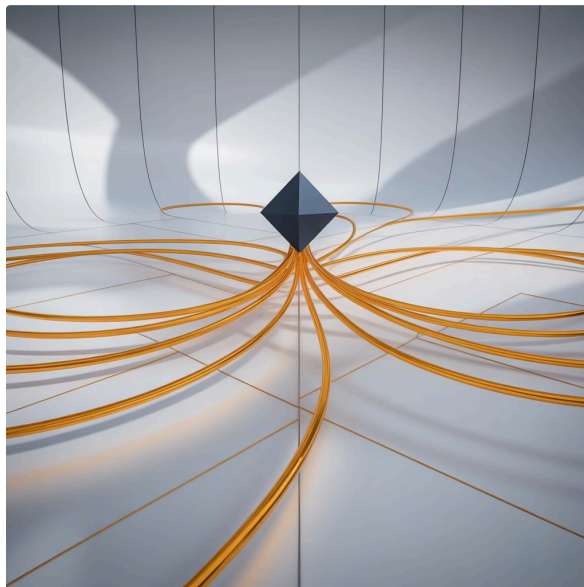
A notação UML para composição é específica e deve ser aplicada corretamente para comunicar a intenção do design de forma clara.

Elementos Visuais

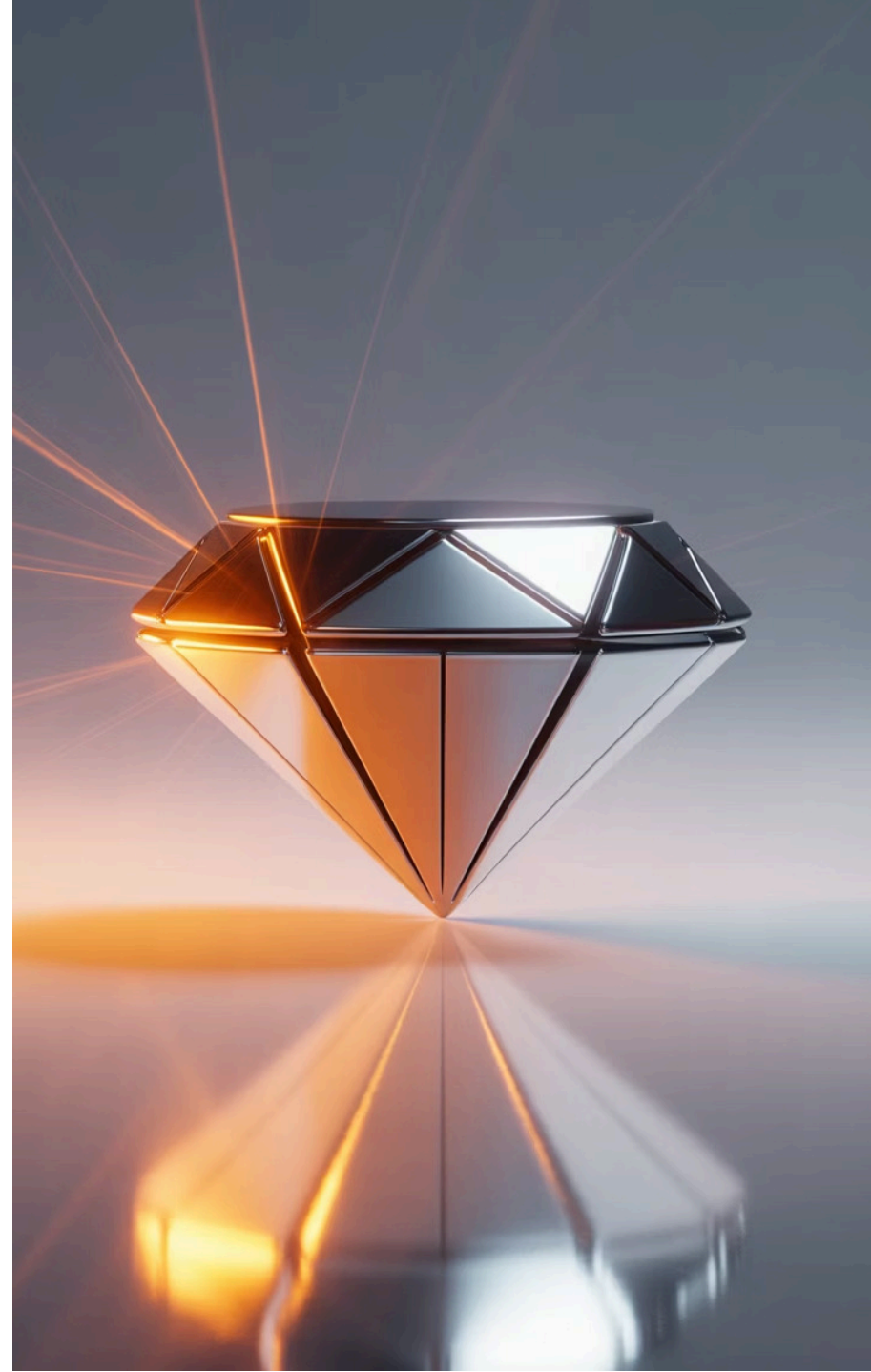
- **Diamante preto** (preenchido) na classe "todo"
- Linha simples conectando as classes
- Multiplicidade nas extremidades
- Nome da associação (opcional)

Significado do Símbolo

O diamante preto indica que a relação é de **composição forte**, diferente do diamante vazio da agregação.



📌 **Dica:** Lembre-se sempre:
diamante preto = composição
forte, diamante vazio =
agregação fraca.



Exemplo Conceitual: Pessoa → Coração

Vamos analisar um exemplo clássico de composição que ilustra perfeitamente o conceito de dependência existencial.



1

Classe Pessoa

Representa o objeto "todo" que possui órgãos internos essenciais para sua existência e funcionamento.

- Controla o ciclo de vida
- Cria os órgãos internos
- Gerencia suas funções

2

Classe Coração

Representa uma parte vital que não pode existir independentemente da pessoa.

- Criado junto com a pessoa
- Funciona apenas dentro do contexto
- Destruído com a pessoa

"O coração não existe sem a pessoa" – este é o princípio fundamental da composição. A parte componente (coração) tem sua existência completamente dependente do todo (pessoa).



Exemplo Conceitual: Carro → Motor

Outro exemplo clássico que demonstra como componentes internos são criados e destruídos junto com o objeto principal.

01

Criação do Carro

Quando um objeto Carro é instanciado, automaticamente cria seu motor interno como parte integral de sua estrutura.

02

Funcionamento Integrado

O motor opera exclusivamente dentro do contexto do carro, não podendo ser utilizado independentemente.

03

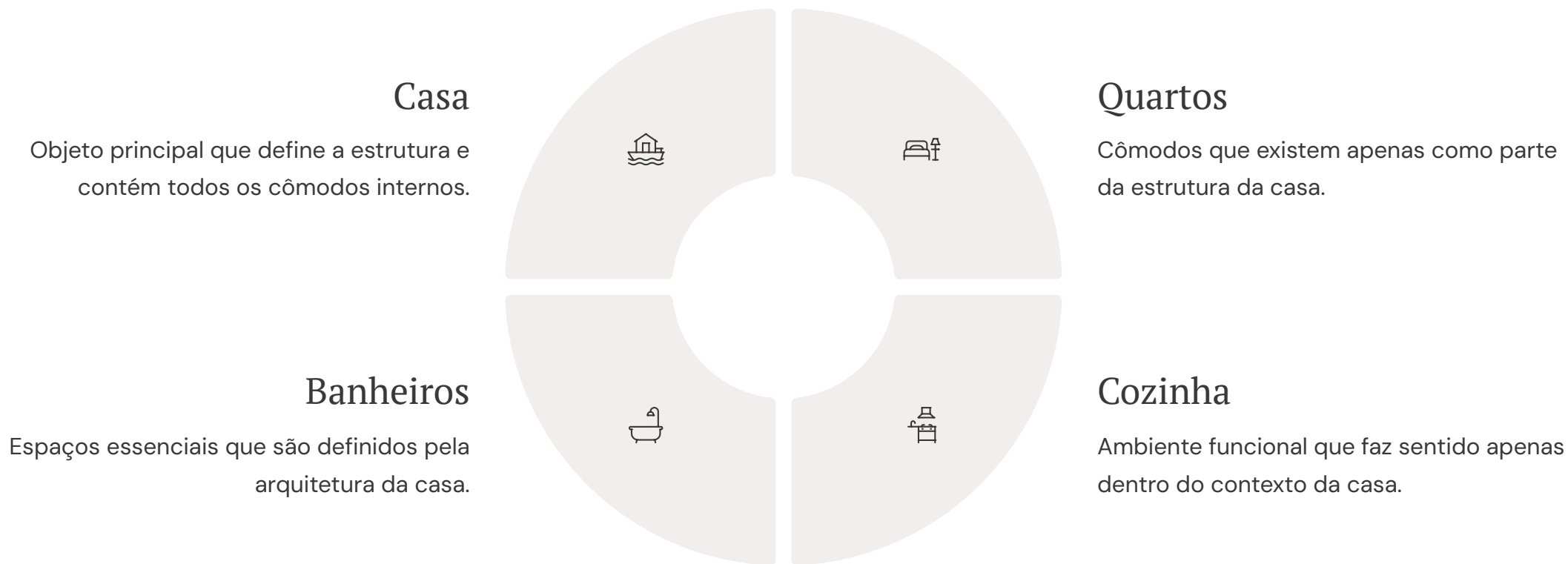
Destruição Conjunta

Quando o carro é destruído ou descartado, seu motor interno é automaticamente eliminado junto.

❏ **Reflexão:** Note que estamos falando de um motor específico criado para aquele carro, não de um motor que pode ser removido e reutilizado em outros veículos.

Exemplo Conceitual: Casa → Cômodos

Este exemplo ilustra como múltiplas partes podem compor um todo, mantendo a relação de dependência existencial.



Os cômodos **não podem existir separados da casa**. Quando a casa é demolida, todos os cômodos deixam de existir automaticamente. Esta é a essência da composição.

PHP – Exemplo 1: Classe Carro com Motor

Vamos implementar o conceito de composição em PHP, demonstrando como o motor é instanciado no construtor e destruído junto com o carro.

```
potencia = $potencia;
$this->tipo = $tipo;
echo "Motor $tipo de $potencia HP foi criado.\n";
}

public function ligar() {
    echo "Motor ligado! Potência: {$this->potencia} HP\n";
}

public function desligar() {
    echo "Motor desligado.\n";
}

public function __destruct() {
    echo "Motor {$this->tipo} foi destruído.\n";
}
}
```

📌 **Observação:** Note que o Motor tem seu próprio construtor e destrutor para demonstrar claramente o ciclo de vida do componente.



PHP – Exemplo 1 (continuação): Demonstração de Construtores e Destrutores

Continuando o exemplo anterior, aqui está a implementação completa da classe Carro que demonstra a composição em ação.

```
marca = $marca;
$this->modelo = $modelo;
// Motor é criado internamente (composição)
$this->motor = new Motor($potenciaMotor, "Gasolina");
echo "Carro $marca $modelo foi montado.\n";
}

public function ligar() {
echo "Ligando $this->marca $this->modelo...\n";
$this->motor->ligar();
}

public function desligar() {
$this->motor->desligar();
echo "Carro desligado.\n";
}

public function __destruct() {
echo "Carro {$this->marca} {$this->modelo} foi destruído.\n";
// Motor será automaticamente destruído
}
}

// Exemplo de uso
$meuCarro = new Carro("Toyota", "Corolla", 140);
$meuCarro->ligar();
$meuCarro->desligar();
unset($meuCarro); // Destrói carro e motor automaticamente
?>
```


PHP – Exemplo 2: Casa e Cômodos

Implementação mais complexa demonstrando composição com múltiplos objetos componentes, onde os cômodos são criados internamente e seu ciclo de vida é controlado pela casa.

```

    nome = $nome;
    $this->area = $area;
    echo "Cômodo '$nome' de {$area}m² foi construído.\n";
}

public function getInfo() {
    return "{$this->nome}: {$this->area}m²";
}

public function __destruct() {
    echo "Cômodo '{$this->nome}' foi demolido.\n";
}
}

class Casa {
    private $endereco;
    private $comodos = []; // Array de cômodos (composição)

    public function __construct($endereco) {
        $this->endereco = $endereco;
        // Cômodos criados internamente (composição)
        $this->comodos[] = new Comodo("Sala", 25);
        $this->comodos[] = new Comodo("Cozinha", 15);
        $this->comodos[] = new Comodo("Quarto", 20);
        $this->comodos[] = new Comodo("Banheiro", 8);
        echo "Casa no endereço '$endereco' foi construída.\n";
    }

    public function listarComodos() {
        echo "Cômodos da casa:\n";
        foreach($this->comodos as $comodo) {
            echo "- " . $comodo->getInfo() . "\n";
        }
    }

    public function __destruct() {
        echo "Casa no endereço '{$this->endereco}' foi demolida.\n";
        // Cômodos serão automaticamente destruídos
    }
}
?>
```

Benefícios da Composição

A aplicação correta da composição traz vantagens significativas para a arquitetura e manutenibilidade do seu código.



Clareza na Modelagem

Torna explícitas as relações de dependência entre objetos, facilitando a compreensão da estrutura do sistema e das responsabilidades de cada componente.

- Relações bem definidas
- Estrutura hierárquica clara
- Intenções de design evidentes



Melhor Encapsulamento

Protege os componentes internos do acesso externo inadequado, garantindo que apenas o objeto "todo" possa manipular suas partes componentes.

- Controle de acesso restrito
- Integridade dos dados
- Redução de acoplamento



Responsabilidades Bem Definidas

Estabelece claramente quem é responsável por criar, gerenciar e destruir cada componente, evitando ambiguidades e problemas de gerenciamento de memória.

- Proprietário único
- Ciclo de vida controlado
- Gerenciamento automatizado



Erros Comuns

Identificar e evitar os erros mais frequentes na implementação de composição é essencial para criar código robusto e bem estruturado.

1

Confundir Composição com Agregação

Tratar uma relação de composição como se fosse agregação, permitindo que partes existam independentemente do todo.

Erro: Criar objetos componentes externamente e apenas referenciá-los.

Solução: Sempre criar componentes internamente no construtor.

2

Não Controlar o Ciclo de Vida

Falhar em garantir que as partes sejam destruídas junto com o todo, causando vazamentos de memória ou inconsistências.

Erro: Não implementar destrutores adequados.

Solução: Usar destrutores e gerenciamento automático de recursos.

3

Permitir Compartilhamento de Partes

Permitir que uma mesma parte pertença a múltiplos objetos "todo", violando o princípio da composição.

Erro: Passar referências de componentes entre objetos.

Solução: Cada parte deve pertencer exclusivamente a um todo.



Aplicações Reais

A composição é amplamente utilizada em diversos domínios da programação, especialmente onde há necessidade de modelar estruturas hierárquicas complexas.



Sistemas Embarcados

Modelagem de componentes de hardware onde sensores, processadores e interfaces são partes integrais do sistema principal. Cada componente só funciona dentro do contexto do sistema completo.



Modelagem de Componentes Internos

Representação de peças e subsistemas em equipamentos industriais, onde cada componente é projetado especificamente para aquele equipamento e não pode ser reutilizado independentemente.



Arquiteturas Orientadas a Objetos Complexas

Frameworks e sistemas empresariais onde módulos internos são criados e gerenciados pelo sistema principal, garantindo coesão e controle centralizado das funcionalidades.



Atividade Prática Orientada

Chegou o momento de aplicar os conhecimentos adquiridos em uma atividade prática que consolidará sua compreensão sobre composição.



Sistema Computador

Modele um computador com seus componentes internos: processador, memória RAM, placa-mãe e fonte de alimentação.

- Componentes criados internamente
- Especificações técnicas
- Métodos de funcionamento



Sistema Corpo Humano

Represente o corpo humano com órgãos vitais: coração, pulmões, fígado e cérebro.

- Órgãos como componentes
- Funções específicas
- Interdependências vitais



Sistema Aplicativo

Desenvolva um aplicativo com módulos internos: interface, banco de dados, autenticação e notificações.

- Módulos especializados
- Configurações integradas
- Gerenciamento centralizado

Objetivo: Escolha um dos sistemas acima e implemente-o demonstrando **3 relações de composição claras** com ciclo de vida controlado e dependência existencial.

Orientações para Atividade

Siga estas orientações específicas para garantir que sua implementação demonstre corretamente os conceitos de composição estudados.

01

Diagramar em UML

Crie um diagrama de classes UML mostrando as relações de composição com diamantes pretos. Identifique claramente qual é o "todo" e quais são as "partes".

- Use diamantes pretos (preenchidos)
- Indique multiplicidades
- Nomeie as associações

02

Implementar em PHP

Desenvolva o código PHP seguindo os padrões demonstrados em aula, com construtores que criam os componentes internamente e destrutores adequados.

- Componentes criados no construtor
- Métodos de interação
- Destrutores implementados

03

Focar em Dependências Fortes

Demonstre claramente que as partes não podem existir sem o todo, implementando controles que evidenciem esta dependência existencial.

- Dependência existencial clara
- Ciclo de vida compartilhado
- Testes de validação

Discussão em Grupo

A fase de discussão é fundamental para consolidar o aprendizado e identificar diferentes abordagens para resolver os mesmos problemas.

Apresentação das Soluções

Cada grupo apresentará sua implementação explicando:

- Justificativa para o sistema escolhido
- Como modelaram as relações de composição
- Decisões de design e implementação
- Demonstração do código funcionando



Comparação de Modelagens

Analisaremos as diferentes abordagens utilizadas pelos grupos para resolver problemas similares, identificando pontos fortes e oportunidades de melhoria em cada solução.

Feedback Construtivo

Forneceremos feedback específico sobre cada implementação, destacando boas práticas aplicadas e sugerindo melhorias técnicas e conceituais.

Resumo da Aula

Vamos consolidar os principais conceitos e conquistas desta aula sobre composição, garantindo que todos os objetivos foram alcançados.

1 — Definição e Representação

Compreendemos que a composição é o relacionamento mais forte entre classes, caracterizada pela dependência existencial total e representada por diamante preto na UML.

2 — Exemplos Práticos e Código

Analisamos exemplos conceituais (Pessoa-Coração, Carro-Motor, Casa-Cômodos) e implementamos soluções práticas em PHP demonstrando o controle de ciclo de vida.

3 — Atividade Aplicada em Grupo

Desenvolvemos e discutimos implementações práticas de sistemas com relações de composição, consolidando o aprendizado através da prática orientada.

📌 **Principais Takeaways:** A composição exige que as partes sejam criadas internamente, tenham ciclo de vida compartilhado e dependência existencial total do objeto "todo".





Próxima Aula (20/10/2025)

A1/4 - Avaliação Parcial A1

Preparem-se para nossa primeira avaliação parcial, que será um momento importante para demonstrar seu domínio dos conceitos estudados até aqui.



Formato da Avaliação

Avaliação individual com questões elaboradas em múltiplos níveis de complexidade para avaliar diferentes aspectos do seu aprendizado.

- Questões conceituais
- Análise e desenvolvimento de código
- Resolução de problemas



Peso e Importância

Esta avaliação tem **peso 10,0** e será fundamental para sua nota parcial no semestre.

- Peso máximo na A1
- Cobertura de todo conteúdo referente as Unidades I e II

Dica de Estudo: Revisem especialmente os pdfs apresentados com os exemplos e exercícios vistos em aula!

Lista de Exercícios – Composição

1. Pessoa e Coração

Modele a classe `Pessoa` que cria e gerencia internamente um objeto `Coracao`. Simule o batimento usando um método.

2. Carro e Motor

Crie uma classe `Carro` que instancia um `Motor` no construtor. Ao destruir o carro, exiba uma mensagem indicando que o motor foi destruído também.

3. Casa e Comodos

A classe `Casa` deve criar uma lista de objetos `Comodo` no construtor. Exiba os nomes dos cômodos ao listar.

4. Computador e Componentes Internos

Modele `Computador` com composição de `PlacaMae`, `Processador` e `MemoriaRAM`. Todos devem ser criados internamente.

5. CorpoHumano e Órgãos

Crie a classe `CorpoHumano` que contém objetos `Coração`, `Pulmão` e `Cérebro`. Simule comportamentos sincronizados.

6. Aplicativo e Módulos

Desenvolva a classe `Aplicativo` que cria internamente diferentes módulos (ex: `Login`, `Dashboard`, `Config`). Liste os módulos ativos.

7. Relógio e Ponteiros

Modele `Relogio` com objetos internos `PonteiroHora`, `PonteiroMinuto`, `PonteiroSegundo`. Simule avanço de tempo.

8. Universidade e Departamentos

A classe `Universidade` deve criar internamente alguns departamentos fixos (`Exatas`, `Humanas`, `Saúde`) e listá-los.

9. BibliotecaDigital e Sessões

Crie `BibliotecaDigital` com composição de várias `Sessao` (`Literatura`, `Ciência`, `História...`). Ao destruir a biblioteca, destrua as sessões.

10. Smartphone e Componentes

Modele `Smartphone` com objetos internos `Tela`, `Bateria`, `Camera`, `Processador`. Todos devem ser criados no construtor e destruídos juntos.